Flyweight

Flyweight · Share glyph styles across a million characters · Build with Waqar

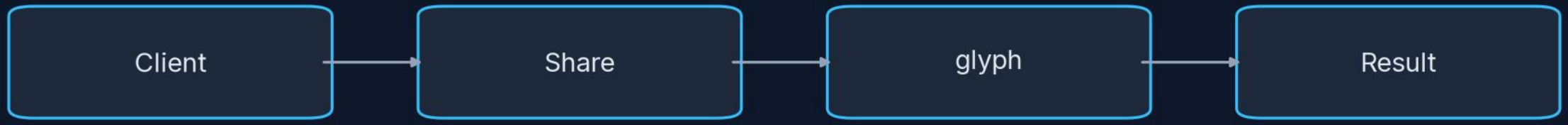
Today's Hook

- Problem: Share glyph styles across a million characters
- Pattern: Flyweight — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Share glyph styles across a million characters
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Flyweight: Share glyph styles across a million characters

Real-World Scenario

- Running example: Share glyph styles across a million characters
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.structural.flyweight;
import com.buildwithwaqar.patterns.gof.structural.flyweight.GlyphFactory;
/** Problem: render a document without allocating a style object per character. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Flyweight problem: shared glyph styles ===");
    var f = new GlyphFactory();
    for (int i = 0; i < 1000; i++) f.glyph((char)('A' + (i % 26)), f.style("Inter", 14, "#fff"), i, 0);
    System.out.println("chars=1000 pooledStyles=" + f.pooled());
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.structural.flyweight;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

record GlyphStyle(String font, int size, String color) {}

record Glyph(char character, GlyphStyle style, int x, int y) {}

/** Flyweight factory pools intrinsic glyph styles. */
public final class GlyphFactory {
    private final Map<GlyphStyle, GlyphStyle> pool = new ConcurrentHashMap<>();

    public GlyphStyle style(String font, int size, String color) {
        var key = new GlyphStyle(font, size, color);
        return pool.computeIfAbsent(key, k -> k);
    }

    public Glyph glyph(char c, GlyphStyle style, int x, int y) {
        return new Glyph(c, style, x, y);
    }
```

Solution (2)

```
    public int pooled() { return pool.size(); }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.gof.structural.flyweight;
import com.buildwithwaqar.patterns.gof.structural.flyweight.GlyphFactory;
/** Problem: render a document without allocating a style object per character. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Flyweight problem: shared glyph styles ===");
    var f = new GlyphFactory();
    for (int i = 0; i < 1000; i++) f.glyph((char)('A' + (i % 26)), f.style("Inter", 14, "#fff"), i, 0);
    System.out.println("chars=1000 pooledStyles=" + f.pooled());
  }
}
```

Walkthrough Output

```
=== Flyweight problem: shared glyph styles ===
chars=1000 pooledStyles=1
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Flyweight.

Related Patterns

- Contrast Flyweight with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Flyweight → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.